

ISM3232 — Week 16 Lab

GenAI Feature & Final Demo — Instructor Facilitation Guide

ISM3232 · Business Application Development · USF Muma College of Business

Module 16 · Unit 4 · Capstone Build

Contents

1 Session Snapshot	2
2 Learning Objectives	2
3 Before Class — Setup Checklist	3
4 Materials Needed	3
5 Timing Plan (75 minutes)	3
6 Segment-by-Segment Walkthrough	5
6.1 Intro (0:00–0:04)	5
6.2 Part 1 — Build <code>ai_feature.py</code> (0:04–0:16, 12 min)	5
6.3 Part 2 — Wire the Feature into <code>app.py</code> (0:16–0:36, 20 min)	8
6.4 Part 3 — Mocked Test and README (0:36–0:50, 14 min)	11
6.5 Part 4 — Final Checklist and Push (0:50–0:58, 8 min)	14
6.6 Part 5 — Final Demo Presentations (0:58–1:15, 17 min, scale to section size)	15
7 Closing the Semester	17
8 Appendix A — Full Answer Key (<code>ai_feature.py</code> + corrected <code>app.py</code> wiring + <code>tests/test_ai_feature.py</code>)	17
9 Appendix B — Extra Practice (only if a student finishes the checklist early, before presentations)	19

1 Session Snapshot

Course	ISM3232 — Business Application Development
Session	Week 16 Lab — GenAI Feature & Final Demo
Unit	Unit 4 · Capstone Build
Class length	Full class period (75 minutes) — the final session of the semester
Format	Live code-along (Parts 1–3), independent polish (Part 4), live presentations (Part 5)
Prerequisites	Week 15: complete, tested, five-tab Streamlit app wired to <code>database.py</code>
Student-facing lab page	Week 16 In-Class Lab — Module 7E & 7F, “GenAI Feature + Final Demo”
Parts covered	Part 1 (<code>ai_feature.py</code>) – Part 4 (final checklist + push), Part 5 (demo presentations)
Submission	2 screenshots, GitHub URL, Canvas, completion credit, live presentation

This is the final lab of the semester, and the lab page’s own warning is worth reading to the class exactly as written: **all six GenAI controls are required, and missing any one fails the feature on the capstone rubric. Read this guide’s Part 2 before class — the lab page’s own provided wiring code, exactly as written, contains a genuine, verified bug in the single most safety-critical control of the six: the human-review-before-save gate does not actually work as written**, due to a classic Streamlit state-management pitfall (a button nested inside another button’s conditional block). This guide explains the bug, why it happens, and ships a verified, working fix using `st.session_state` — treat this the way Week 5’s floating-point/chained-comparison bugs were treated: a genuine teaching opportunity, not something to quietly route around.

2 Learning Objectives

By the end of this class period, students should be able to:

1. Call the Anthropic API from Python, with a prompt containing only the minimum data needed — no personally identifiable information.
2. Explain and implement all six required GenAI controls: disclosure, human review, no-PII, mocked testing, documented limitations, and full code comprehension.
3. Understand why `st.session_state` is necessary to persist data (like an AI-generated summary) across Streamlit’s automatic script re-runs, and why a naive nested-button pattern fails.

4. Write a `pytest` test that mocks an external API call, verifying application logic without making a real network request or requiring a real API key.
5. Present a complete, working capstone project, covering business problem, developer workflow, OOP design, SQL, the live interface, the GenAI feature, testing, and an honest AI-use disclosure.

3 Before Class — Setup Checklist

- ☐ **Critical: read this guide's Part 2 in full before class, and rehearse the corrected `st.session_state` version yourself.** The lab page's provided "Looks good — save with this summary" button, nested inside the "Generate AI Summary" button's `if` block, does not fire correctly on the very next interaction — verified directly with Streamlit's own `AppTest` simulation tool, detailed fully below. Decide whether to demonstrate the bug live (recommended, matching this course's established debugging culture) or present the corrected version directly.
- ☐ Obtain a real Anthropic API key before class if you intend to demonstrate Part 1's live API call — set it as the `ANTHR0PIC_API_KEY` environment variable; without it, `anthropic.Anthropic()` raises an authentication error the moment a real (non-mocked) call is attempted, verified below.
- ☐ Plan Part 5's presentation logistics in advance — with a full class of students each presenting eight required topics, time management is the real challenge; decide and communicate a firm per-student time limit (this guide assumes roughly 5 minutes per student as a starting point, adjustable to your actual section size) before the period begins.
- ☐ This is the last class of the semester — budget a genuine moment for it at the end, separate from the mechanics of any single student's demo.

4 Materials Needed

- Terminal (zsh), VS Code, Python 3.10+, the existing `module07_final_project` venv with a complete, working `app.py` and `database.py` from Weeks 14–15
- An Anthropic API key (`ANTHR0PIC_API_KEY` environment variable) for live demonstration only — not required for students' own mocked tests to pass

5 Timing Plan (75 minutes)

Time	Segment	Minutes
0:00–0:04	Welcome: "the six required controls, and the last lab"	4
0:04–0:16	Part 1 — Build <code>ai_feature.py</code>	12

Time	Segment	Minutes
0:16–0:36	Part 2 — Wire the feature into <code>app.py</code> (including the button-state bug)	20
0:36–0:50	Part 3 — Mocked test and README	14
0:50–0:58	Part 4 — Final checklist and push	8
0:58–1:15	Part 5 — Final demo presentations	17

Part 2 receives extra time specifically to properly diagnose and fix the button-nesting issue rather than rushing past it; Part 5’s actual duration will depend heavily on section size — treat the 17 minutes shown as a starting allocation for a small section or a first wave of presenters, and plan realistically for a larger class (see Part 5’s facilitation notes on scaling this beyond one class period if needed).

6 Segment-by-Segment Walkthrough

6.1 Intro (0:00–0:04)

Say to the class:

“Last lab of the semester. Today you add one final feature to your capstone — a controlled AI integration — and then present the whole thing. Six controls are required for the AI feature, and I want to name all six right now, because missing even one fails this specific rubric item: a disclosure label, a human review step before anything AI-generated gets saved, no personal information sent to the API, a test using a mocked API call, documented limitations in your README, and — the one that matters most — you personally being able to explain every single line, without notes.”

Do: Write the six controls on the board, numbered, and leave them visible through Part 3.

6.2 Part 1 — Build `ai_feature.py` (0:04–0:16, 12 min)

Teaching goal: A single, narrowly-scoped function calling the Anthropic API — deliberately minimal in what data it sends, setting up Control #3 (no PII) before any UI wiring happens.

Say to the class:

“One function, one job: take a category and a description, return a one-sentence summary. Notice what’s not a parameter here — no name, no ID, no dollar amount. That’s deliberate, and it’s Control #3 built in from the very first line, not bolted on afterward.”

Live-code this:

```
cd ~/ism3232/module07_final_project
source .venv/bin/activate
pip install anthropic
pip freeze > requirements.txt
touch ai_feature.py && code ai_feature.py
```

```
import anthropic

def summarise_request(category: str, description: str) -> str:
    """
    Return a one-sentence AI summary of a business request.
    Only category and description are sent -- no names, IDs, or amounts.
    """
    client = anthropic.Anthropic()
```

```

prompt = f'Summarise this {category} business request in one clear sentence:\n{description}'
message = client.messages.create(
    model='claude-haiku-4-5-20251001',
    max_tokens=150,
    messages=[{'role': 'user', 'content': prompt}]
)
return message.content[0].text

```

Line-by-line explanation:

- `def summarise_request(category: str, description: str) -> str:` — **exactly two parameters, both business-relevant, neither identifying a specific person.** Say explicitly: this function signature *is* Control #3 — a requester name parameter was never added in the first place, meaning there's no code path by which it could accidentally leak into a prompt later. Preventing a class of mistake structurally, rather than remembering to avoid it every time, is worth naming as a genuinely good engineering habit.
- The docstring's second line — "Only category and description are sent -- no names, IDs, or amounts." — worth stating explicitly: **a docstring stating a security/privacy property is itself part of good practice** — it documents an intentional design constraint for any future reader (including the student themselves, months from now).
- `client = anthropic.Anthropic()` — creates a client object; by default, this reads the API key from the `ANTHROPIC_API_KEY` environment variable automatically — say explicitly, no key is hardcoded anywhere in this file, which matters for the same reason `.gitignore` excludes `.venv/`: a hardcoded key would end up committed to a public-facing GitHub repo, a genuine, common real-world security mistake.
- `prompt = f'Summarise this {category} business request in one clear sentence:\n{description}'` — an f-string building the actual instruction sent to the model — note `\n` inside the f-string, a literal newline character separating the instruction from the description text.
- `client.messages.create(model=..., max_tokens=150, messages=[...])` — the actual API call; `max_tokens=150` caps how long the response can be, appropriate for a one-sentence summary; `messages=[{'role': 'user', 'content': prompt}]` is the Anthropic API's standard message format — worth a brief note that this exact shape (a list of role/content dictionaries) is a common convention across most current LLM APIs, not unique to this one.
- `return message.content[0].text` — extracts just the generated text from the API's full response object — say explicitly, the response contains more structure than just the text (metadata, usage stats), and this line intentionally returns only the part the rest of the application needs.

Test it manually, exactly as the lab page specifies (requires a real `ANTHROPIC_API_KEY` to succeed):

```
python3
>>> from ai_feature import summarise_request
>>> result = summarise_request('Travel', 'Flight and hotel for Q4 sales conference')
>>> print(result)
>>> exit()
```

If a student's ANTHROPIC_API_KEY isn't set, verified exact failure:

```
TypeError: Could not resolve authentication method. Expected one of api_key,
auth_token, or credentials to be set. Or for one of the `X-API-Key` or
`Authorization` headers to be explicitly omitted
```

Say explicitly if this comes up: “This isn’t a bug in your code — it’s the SDK correctly refusing to make a request with no credentials. Route students without a working key to whatever your course’s shared key-provisioning process is; don’t let this block Part 1 for long, since Part 3’s tests don’t need a real key at all.”

Common student mistakes to watch for:

- Adding requester (or any other identifying field) as a third parameter “just in case it’s useful for the summary” — redirect firmly back to Control #3; if a genuinely business-relevant detail seems missing, the fix is expanding description itself, written by the user, not adding new structured PII fields.
- Hardcoding an API key directly in the file instead of relying on the environment variable — flag this immediately and seriously if you see it, given the real security consequences of a committed key.

Check for understanding: “If this function’s docstring didn’t include that second line about what’s sent, would the function’s actual behavior be any different?” (No — the docstring is documentation, not enforcement; the actual privacy guarantee comes from the function signature genuinely not accepting a name/ID/amount parameter at all. A good check that students distinguish *documenting* a property from *guaranteeing* it structurally.)

6.3 Part 2 — Wire the Feature into app.py (0:16–0:36, 20 min)

Teaching goal: Add the AI feature to Tab 1 with disclosure and human review — and confront, directly, a genuine bug in the lab’s own provided wiring code.

Say to the class:

“I’m going to type this exactly as given first, and then we’re going to test the review step specifically, carefully — because I want you to see something for yourselves before I explain it.”

Live-code this, added to Tab 1 in app.py, after the existing form fields:

```
# AI summary feature
description = st.text_area('Request description (for AI summary)')

if description:
    if st.button('Generate AI Summary'):
        with st.spinner('Generating summary...'):
            from ai_feature import summarise_request
            ai_summary = summarise_request(category, description)
            # Required: disclosure label
            st.info(f'AI-generated summary (review before saving):\n\n{ai_summary}')
            st.caption('AI output generated by Claude. Review before saving.')

            # Required: human review gate
            if st.button('Looks good -- save with this summary'):
                add_record(requester, category, amount, notes=ai_summary)
                st.success('Record saved with AI summary.')
```

Run it, enter a description, click “Generate AI Summary.” The summary appears correctly, with the disclosure caption — Controls #1 (disclosure) and, apparently, #2 (human review) both look present.

Now — click “Looks good — save with this summary.”

Say explicitly, before revealing what happens: “Watch closely. Predict what you’ll see.”

Verified, actual behavior: nothing saves. No success message appears. **The “Looks good” button itself disappears from the screen entirely.** No error, no crash — the click simply has no effect, and the review button vanishes.

Explain why, precisely, since this is the lab’s central technical lesson today:

“Remember Week 15’s core fact: Streamlit re-runs the entire script, top to bottom, on every interaction — including this click. When you clicked ‘Generate AI Summary,’ that specific click

made `st.button('Generate AI Summary')` return `True` for that one run — and inside that run, the whole block executed: the summary generated, the info box shown, and — critically — the ‘Looks good’ button itself got defined, for the first time, as part of that same run. But clicking ‘Looks good’ triggers a **new, separate re-run** of the entire script from the top. On this new run, `st.button('Generate AI Summary')` is evaluated fresh again — and since this click was on the ‘Looks good’ button, not ‘Generate AI Summary,’ that condition is `False` this time. Which means the **entire `if st.button('Generate AI Summary'):` block — including the line that defines the ‘Looks good’ button in the first place — never runs on this pass.** The click on a button that, from Streamlit’s perspective, technically no longer exists on this run has nowhere to register. That’s why it vanishes and nothing saves.”

Say explicitly, and this is worth landing seriously, given what this control actually is: “This is the *human review gate* — Control #2, one of the six required, and arguably the most safety-critical one, since it’s the last checkpoint before AI-generated text gets permanently saved. As literally written, in the lab’s own provided code, it does not work. This is exactly the kind of bug this course has trained you to catch — read the code, understand the model it’s built on (Streamlit’s re-run behavior), and don’t just trust that code labeled ‘required’ automatically does what its comment says.”

Now, the fix — using `st.session_state` to persist the summary across re-runs:

```
# AI summary feature
description = st.text_area('Request description (for AI summary)')

if description:
    if st.button('Generate AI Summary'):
        with st.spinner('Generating summary...'):
            from ai_feature import summarise_request
            st.session_state['ai_summary'] = summarise_request(category, description)

    if 'ai_summary' in st.session_state:
        # Required: disclosure label
        st.info(f"AI-generated summary (review before saving):\n\n{st.session_state['ai_summary']}")
        st.caption('AI output generated by Claude. Review before saving.')

        # Required: human review gate
        if st.button('Looks good -- save with this summary'):
            add_record(requester, category, amount, notes=st.session_state['ai_summary'])
            st.success('Record saved with AI summary.')
            del st.session_state['ai_summary']
```

Line-by-line explanation of the fix:

- `st.session_state` — **new syntax, and the direct solution to the problem just diagnosed.** Say explicitly: `st.session_state` is a dictionary-like object that **persists across re-runs**, for as long as a user’s browser session stays open — unlike ordinary Python variables in the script (which are recreated fresh on every single re-run and forgotten immediately afterward), anything stored in `st.session_state` survives.
- `st.session_state['ai_summary'] = summarise_request(...)` — instead of storing the summary in a plain local variable (which would vanish the instant this re-run ends), it’s stored in session state — meaning it’s still there on the *next* re-run, even though that next re-run is triggered by a completely different button click.
- `if 'ai_summary' in st.session_state:` — **this check now lives *outside* the `if st.button('Generate AI Summary')` block**, at the same indentation level, not nested inside it. This is the structural fix: whether to show the summary and the review button no longer depends on *this specific run* being the one where ‘Generate’ was clicked — it depends only on whether a summary is currently *stored*, which persists correctly across the follow-up re-run triggered by clicking ‘Looks good.’
- `del st.session_state['ai_summary']` — after a successful save, the stored summary is explicitly removed — say explicitly why this matters: without it, the summary and review button would keep reappearing on every future re-run indefinitely, even for a completely new, unrelated submission.

Re-run the corrected version and re-test the full flow. Verified: the success message now appears correctly after clicking “Looks good.”

Common student mistakes to watch for:

- Copying the lab’s provided nested-button version verbatim, testing only “does the summary generate” (which works fine) without specifically testing the *save* step — this is worth stating explicitly as the reason the bug can go unnoticed: the first half of the flow works perfectly, and only clicking all the way through to save reveals the problem.
- After applying the `session_state` fix, forgetting the `del st.session_state['ai_summary']` cleanup line — not a functional bug exactly, but produces a confusing UX where an old summary keeps showing up unexpectedly.

Check for understanding: “Why didn’t the *first* required control — the disclosure label — have this same bug, but the *second* one — human review — did?” (Because the disclosure label (`st.info(...)`) is defined and shown within the *same* re-run as clicking ‘Generate’ — it doesn’t need to persist to a *different*, later re-run the way the review button’s click-and-save action does. Anything that needs to survive across two separate user interactions needs `st.session_state`; anything that only needs to exist within one single interaction’s re-run doesn’t.)

6.4 Part 3 — Mocked Test and README (0:36–0:50, 14 min)

Teaching goal: A pytest test that verifies the AI feature’s logic **without** making a real API call — Control #4 — using `unittest.mock`.

Say to the class:

“Testing code that calls a real, paid, network-dependent API is a genuine problem — you don’t want your test suite to cost money or fail because of network issues, and you don’t want tests that only pass if everyone has a valid API key. The fix: mock the API call entirely, replacing it with a stand-in that returns a fake, predictable response.”

Live-code this:

```
# tests/test_ai_feature.py
from unittest.mock import MagicMock, patch
from ai_feature import summarise_request

def test_summarise_returns_string():
    with patch('ai_feature.anthropic.Anthropic') as mock_client:
        mock_message = MagicMock()
        mock_message.content = [MagicMock(text='AI summary.')]
        mock_client.return_value.messages.create.return_value = mock_message
        result = summarise_request('Travel', 'Flight to Atlanta for client meeting')
        assert isinstance(result, str)
        assert len(result) > 0

def test_prompt_does_not_include_name():
    with patch('ai_feature.anthropic.Anthropic') as mock_client:
        mock_message = MagicMock()
        mock_message.content = [MagicMock(text='Short summary.')]
        mock_client.return_value.messages.create.return_value = mock_message
        result = summarise_request('Software', 'License renewal for design tools')
        assert 'Taylor' not in str(result)
        assert 'Jordan' not in str(result)
```

Line-by-line explanation:

- `from unittest.mock import MagicMock, patch` — `unittest.mock` is part of Python’s standard library, no installation needed — new tools this lab introduces for the first time.
- `with patch('ai_feature.anthropic.Anthropic') as mock_client:` — **this is the core mechanism, worth explaining precisely.** `patch(...)` temporarily **replaces** the real `anthropic.Anthropic` class, *as it’s referenced inside `ai_feature.py`* (note the string `'ai_feature.anthropic.Anthropic'` — patching where a thing is *used*, not just where it’s originally defined, is a genuine, important detail of how `patch` works correctly), with a fake

stand-in for the duration of the with block only.

- `mock_message = MagicMock()` then `mock_message.content = [MagicMock(text='AI summary.')] — building a fake response object that mimics the real API's shape closely enough for ai_feature.py's message.content[0].text line to work correctly against it — say explicitly, this fake object doesn't need to be a perfect replica of the real API response, only close enough in the specific parts the code under test actually reads.`
- `mock_client.return_value.messages.create.return_value = mock_message` — configures the mock so that, when `ai_feature.py`'s code calls `client.messages.create(...)`, it receives `mock_message` back, instead of making any real network request.
- `result = summarise_request(...)` — **called completely normally** — this is the payoff of good mocking: the function under test doesn't know or care that it's talking to a fake client instead of a real one; the test exercises the *real* `summarise_request` logic, just with a substituted API layer.
- `test_prompt_does_not_include_name` — say explicitly, this test's *name* promises more than its current assertions actually check: it only confirms specific literal names ('Taylor', 'Jordan') don't appear in the **returned mock text** — which, since the mock always returns a fixed fake string regardless of input, doesn't really test the *prompt sent to the API* at all. A stronger version would inspect `mock_client.return_value.messages.create.call_args` (visible but unused in the lab's own code) to verify what was actually sent — worth mentioning as a genuine, honest limitation of this specific test, and a good target for Appendix B's extra practice.

Run it:

```
pytest -v
```

Verified output — both pass, with no real API key required, no network call made:

```
tests/test_ai_feature.py::test_summarise_returns_string PASSED
tests/test_ai_feature.py::test_prompt_does_not_include_name PASSED
2 passed
```

Now, add the required AI feature section to `README.md`:

```
## AI Feature

**What it does:** [one sentence]

**Controls implemented:**
- AI-generated content is labelled clearly in the UI
- Human review required before any AI output is saved
- No personally identifiable information is sent to the API

**Limitations:**
- [limitation 1]
```

– [limitation 2]

****AI use during development:****

[Describe honestly what AI tools you used and how]

Facilitation note on “Limitations”: push students toward genuine, specific limitations, not vague hedging — good examples: “the summary quality depends entirely on how detailed the user’s description is,” “there’s no retry logic if the API call fails or times out,” “the summary is not validated for factual accuracy against the original description.” A generic “AI can make mistakes” is technically true but not a specific enough limitation for this deliverable.

Common student mistakes to watch for:

- Testing only the “happy path” (a normal-looking description) and not considering what `summarise_request` does with an empty or very long description — not required by today’s two tests, but worth a passing mention as the kind of edge case a more thorough test suite would cover.
- An AI Use Statement that’s vague rather than specific, echoing every prior week’s disclosure requirement — hold the same standard as always.

6.5 Part 4 — Final Checklist and Push (0:50–0:58, 8 min)

Teaching goal: A comprehensive, explicit final review before the capstone is considered complete — the last checkpoint before Part 5’s live presentation.

Say to the class:

“Every item on this list, checked honestly, before you present. I will ask about anything that looks unchecked.”

Work through the full checklist together, on screen:

- All 5 database functions pass tests.
- All 5 Streamlit features work end-to-end.
- GenAI feature shows disclosure label and human review button.
- README complete with AI feature section and limitations.
- requirements.txt current from pip freeze.
- .gitignore includes .venv/, __pycache__/, *.db, .env.
- ruff format and ruff check pass.
- All tests pass.
- Repository accessible to instructor.
- Screenshots/ folder has at least 2 images of the running app.
- You can explain every file and every function without reading notes.

Facilitation notes on the two items most worth double-checking personally, as you circulate:

- **“GenAI feature shows disclosure label and human review button”** — given Part 2’s bug, specifically re-test the *save* step, not just the *generate* step, for every student — a summary that generates correctly but never actually saves would incorrectly read as “working” on a quick glance.
- *****.gitignore includes .venv/, *pycache*/, *.db, .env***** — note .env is new to this specific checklist, not mentioned explicitly in earlier weeks’ .gitignore instructions — worth a brief note if a student asks why: a .env file is a common convention for storing an API key locally (outside this lab’s specific os.environ-based approach, but worth knowing about) — excluding it preemptively is good practice even if today’s ai_feature.py doesn’t use one directly.

Now the final push:

```
ruff format . && ruff check . && pytest -v
git add . && git commit -m 'lab 16: GenAI feature + final submission' && git push
```

Common student mistakes to watch for:

- Treating the checklist as a formality and checking boxes without genuinely re-verifying — this is worth being direct about: this is the actual, final review before public presentation, worth taking seriously rather than rushing through in the last few minutes.

6.6 Part 5 — Final Demo Presentations (0:58–1:15, 17 min, scale to section size)

Teaching goal: Each student presents their complete capstone, covering all eight required topics — the culmination of the entire semester.

Say to the class:

“Eight topics, in order. I will stop you if you skip one — not to be strict for its own sake, but because each of these eight represents a real, distinct part of what you built this semester, and a strong presentation shows all eight, not just the flashiest parts.”

State the eight required topics explicitly, exactly as the lab page specifies:

#	Topic	What to show
1	Business problem + user workflow	Describe the problem and walk through a user session
2	Developer workflow	Terminal: venv active, ruff passing, pytest green, git log
3	OOP design	Walk through <code>models.py</code> : classes, attributes, methods
4	SQL schema + key queries	<code>CREATE TABLE</code> , a filter query, the <code>GROUP BY</code> report
5	Streamlit demo	Live: submit, view, filter, update status, show report
6	GenAI feature	Show disclosure label, review step, explain the prompt
7	Testing + limitations	pytest passing, name one limitation
8	AI use statement	Honest account of what AI tools were used and how

Facilitation notes on running presentations for an entire section in one class period:

- **Set and announce a firm per-student time limit before starting** — with eight topics each deserving genuine coverage, 5 minutes per student is a reasonable floor; do the arithmetic against your actual section size before class and decide honestly whether one 75-minute period accommodates everyone, or whether presentations need to split across today and a follow-up session (office hours, an asynchronous video submission, or a second class meeting if your course calendar allows it) — this is worth planning explicitly rather than discovering mid-class that time has run out for the last several students.

- **Topic 2 (developer workflow)** is a genuinely good “does this actually work” check, live, in front of you — a student whose `ruff` check or `pytest` doesn’t actually pass cleanly when run live, despite claiming it does in their checklist, is worth noting directly and kindly, as useful, honest feedback rather than a gotcha.
- **Topic 6 (GenAI feature)** is the most likely place to see whether Part 2’s bug was genuinely caught and fixed — watch specifically for whether the student demonstrates the *full* flow (generate, then save) or only the generate step; if only the generate step is shown, that’s worth a direct, specific question.
- **Topic 8 (AI use statement)** deserves the same seriousness as every prior week’s disclosure — a strong closing statement is specific about which tools were used, for what, and reflects genuinely on whether that use built or shortcut the student’s own understanding, exactly the framing established since the very first AI-literacy exercise of the semester.

Common issues to watch for during presentations:

- A student skipping straight to the “impressive” parts (the live Streamlit demo, the AI feature) and rushing or omitting the less visually interesting ones (the OOP design walkthrough, the developer workflow) — this is exactly what the “I will stop you if you skip one” framing exists to prevent; enforce it consistently across all students, not selectively.
- Presentations that read code silently rather than explaining it aloud — a good, gentle redirect: “tell me what this class does, in your own words, as if I’ve never seen it” — directly testing the final checklist item (“can explain every file and function without reading notes”).

7 Closing the Semester

There is no “preview next week” for this guide — Week 16 is the end of the course. A few things worth saying to the room explicitly, in whatever words feel natural to you, before the period ends:

- **Name the arc explicitly:** “Week 1, you checked whether your terminal prompt showed % or \$. Today, you presented a working, tested, database-backed, AI-integrated web application, built entirely by you, that you can explain line by line. That distance is worth acknowledging directly.”
- **The debugging and honesty habits matter beyond this course:** the rubber-duck debugging, the AI-use disclosures, the boundary-case tests, the pre-submission ritual — these are genuine professional habits, not classroom-specific requirements, worth naming as such on the way out.
- **If time allows,** a genuine closing question to the room: “What’s one thing from this semester you didn’t expect to still be using by Week 16?” — a good, low-stakes way to let the semester’s actual learning surface in students’ own words, rather than only in a checklist.

8 Appendix A — Full Answer Key (ai_feature.py + corrected app.py wiring + tests/test_ai_feature.py)

```
# ai_feature.py
import anthropic

def summarise_request(category: str, description: str) -> str:
    """
    Return a one-sentence AI summary of a business request.
    Only category and description are sent -- no names, IDs, or amounts.
    """
    client = anthropic.Anthropic()
    prompt = f'Summarise this {category} business request in one clear sentence:\n{description}'
    message = client.messages.create(
        model='claude-haiku-4-5-20251001',
        max_tokens=150,
        messages=[{'role': 'user', 'content': prompt}]
    )
    return message.content[0].text
```

Corrected app.py Tab 1 addition (uses st.session_state — see Part 2 for why the lab’s originally-provided nested-button version does not work):

```

# AI summary feature
description = st.text_area('Request description (for AI summary)')

if description:
    if st.button('Generate AI Summary'):
        with st.spinner('Generating summary...'):
            from ai_feature import summarise_request
            st.session_state['ai_summary'] = summarise_request(category, description)

    if 'ai_summary' in st.session_state:
        # Required: disclosure label
        st.info(f"AI-generated summary (review before saving):\n\n{st.session_state['ai_summary']}")
        st.caption('AI output generated by Claude. Review before saving.')

        # Required: human review gate
        if st.button('Looks good -- save with this summary'):
            add_record(requester, category, amount, notes=st.session_state['ai_summary'])
            st.success('Record saved with AI summary.')
            del st.session_state['ai_summary']

```

```

# tests/test_ai_feature.py
from unittest.mock import MagicMock, patch
from ai_feature import summarise_request

def test_summarise_returns_string():
    with patch('ai_feature.anthropic.Anthropic') as mock_client:
        mock_message = MagicMock()
        mock_message.content = [MagicMock(text='AI summary.')]
        mock_client.return_value.messages.create.return_value = mock_message
        result = summarise_request('Travel', 'Flight to Atlanta for client meeting')
        assert isinstance(result, str)
        assert len(result) > 0

def test_prompt_does_not_include_name():
    with patch('ai_feature.anthropic.Anthropic') as mock_client:
        mock_message = MagicMock()
        mock_message.content = [MagicMock(text='Short summary.')]
        mock_client.return_value.messages.create.return_value = mock_message
        result = summarise_request('Software', 'License renewal for design tools')
        assert 'Taylor' not in str(result)
        assert 'Jordan' not in str(result)

```

Verified end-to-end (via `streamlit.testing.v1.AppTest` with a mocked Anthropic client): generating a summary and clicking “Looks good — save with this summary” against the corrected version produces the success message 'Record saved with AI summary.'; the lab's originally-provided nested-button version, tested identically, produces no success message and the review button silently disappears on the save click.

9 Appendix B — Extra Practice (only if a student finishes the checklist early, before presentations)

Extra — a genuinely stronger test_prompt_does_not_include_name. Have students rewrite the test to actually inspect what was sent to the (mocked) API, rather than only checking the mock's canned return value:

```
def test_prompt_does_not_include_name_stronger():
    with patch('ai_feature.anthropic.Anthropic') as mock_client:
        mock_message = MagicMock()
        mock_message.content = [MagicMock(text='Short summary.')]
        mock_client.return_value.messages.create.return_value = mock_message
        summarise_request('Software', 'License renewal for Taylor\'s design tools')
        call_kwargs = mock_client.return_value.messages.create.call_args.kwargs
        sent_prompt = call_kwargs['messages'][0]['content']
        assert 'Software' in sent_prompt          # category IS sent -- expected
        assert 'Taylor' in sent_prompt           # and so is this, unexpectedly!
        # summarise_request only controls the PROMPT STRUCTURE, not whether a
        # user pastes a name into their own free-text description -- a real,
        # code-cannot-fully-prevent limitation worth naming in the README.
```

This is worth presenting as a genuinely open-ended discussion, not a clean answer: the function's *code* never adds a name, but if a user's own description text happens to *contain* one, that name would legitimately be sent to the API — a real limitation worth naming explicitly in the README's Limitations section, and a good final example of the difference between a control enforced by *code structure* versus one that depends on *user behavior*.